

Python Lessons 7-8 + controller.py Review

Raspberry Pi ENAUTO Python lab - main(), __main__, imports, and function execution

Purpose of This Review

This PDF captures the clean structure built after Lesson 6: moving execution into a main() function, adding the Python __main__ guard, and experimenting with a separate controller.py file that imports and uses functions from lesson8.py.

Core idea: imports bring in tools, definitions create reusable tasks, and the bottom of the script decides what actually executes.

Lesson 7 - The main() Function

Lesson 7 moved the loose action lines at the bottom of the script into one organized main() function. The output stayed the same, but the structure became cleaner and easier to maintain.

Before Lesson 7, the bottom of the script looked like this:

```
available_ram = get_available_ram()
cpu_cores = get_cpu_cores()
```

```
display_device_info()
check_ram(available_ram)
check_cpu(cpu_cores)
lab_ready(available_ram, cpu_cores)
```

After Lesson 7, those actions were wrapped into main():

```
def main():
    available_ram = get_available_ram()
    cpu_cores = get_cpu_cores()

    display_device_info()
    check_ram(available_ram)
    check_cpu(cpu_cores)
    lab_ready(available_ram, cpu_cores)
```

```
main()
```

Mental model: def main() defines the main mission. The line main() at the bottom executes that mission.

Lesson 8 - The __main__ Guard

Lesson 8 replaced the plain main() call with Python's standard direct-run guard. This lets the file be used two different ways: as a script you run directly, or as a module imported by another script.

```
if __name__ == "__main__":
    main()
```

Meaning:

Situation	What Python sets	Result
Run directly: python lesson8.py	__name__ = "__main__"	main() runs
Imported: import lesson8	__name__ = "lesson8"	functions load, but main() does not auto-run

Plain English: the __main__ guard is a safety switch. It prevents lesson8.py from dumping its full output just because another file imported it.

controller.py - Importing lesson8 and Calling One Function

The experiment with controller.py was meant to pull a specific value from lesson8.py without running the whole Lesson 8 script. That is exactly why the `__main__` guard matters.

Clean controller.py version:

```
import lesson8
```

```
available_ram = lesson8.get_available_ram()
```

```
print("Available RAM:", available_ram, "GiB")
```

Expected output:

Available RAM: 7.4 GiB

What each line does:

Line	Meaning
<code>import lesson8</code>	Bring in the lesson8.py file as a module/toolbox.
<code>lesson8.get_available_ram()</code>	Go into lesson8 and run the <code>get_available_ram</code> function.
<code>available_ram = ...</code>	Store the returned number in a variable.
<code>print(...)</code>	Display the value for the user.

The Big Function Lesson

The key difference discovered during the controller.py experiment was function reference versus function execution.

```
lesson8.get_available_ram    # points to the function
lesson8.get_available_ram()  # runs the function and returns the value
```

No parentheses means Python is looking at the function object. Parentheses mean Python executes the function.

Common Errors From the Experiment

Error / Output	What it meant	Fix
<code>NameError: available_ram is not defined</code>	The variable was printed before it was created.	Create it first: <code>available_ram = lesson8.get_available_ram()</code>
<code>SyntaxError: expected '('</code>	A def line was missing parentheses.	Use: <code>def function_name():</code>
<code>SyntaxError: expected ':'</code>	A function definition was missing the colon.	End the def line with <code>:</code>
	Python printed the function object, not the result.	Use parentheses to execute the function.

Final Pattern to Remember

```
# 1. Import tools or modules
import lesson8

# 2. Run a function from the imported module
available_ram = lesson8.get_available_ram()

# 3. Use the returned value
print("Available RAM:", available_ram, "GiB")
```

Full mental model:

Part	Plain English
import lesson8	Bring the toolbox into the current script.
lesson8.get_available_ram	Point at the tool inside that toolbox.
lesson8.get_available_ram()	Run the tool and get the returned value.
def main():	Define the main mission of a script.
if __name__ == "__main__":	Only run the mission when this file is launched directly.

Checkpoint Before the Next Lesson

At this point, the current project has three important learning artifacts:

File	Purpose
lesson7.py	Introduces main() to organize the execution section.
lesson8.py	Adds the __main__ guard so the script can be safely imported.
controller.py	Imports lesson8 and calls one function from it.

Next logical step: turn controller.py into its own clean main() script that imports multiple functions from lesson8.py and prints a smaller custom report.